A1 — Multi-Armed Bandits: task, goal, and how it was solved

A written explanation of what phase A1 asks, why the ladder starts here, how the three algorithms differ, and what the runs actually showed. The lab notebook with the numbers is `experiments/a1_bandits/README.md`; this document is the reasoning behind it.

> **At a glance**
>
> - Thompson sampling has the least regret after 1,000 pulls (71.5 ± 33), ahead of UCB1 at c = 2 (153) and ε-greedy at 0.1 (226) — and a quarter of ε-greedy's spread across seeds.
> - Both exploration knobs trace an inverted U. The best values are ε ≈ 0.04 and c ≈ 1; the textbook c = 2 costs twice as much at this horizon.
> - On a drifting bandit a constant step size finds the best arm 74% of the time against 45% for sample averages, and the sample-average failure is stale information, not variance.

---

1. The task

`ladder.md` states three items:

1. Implement ε-greedy, UCB1, and Thompson sampling on a synthetic k-armed bandit; plot regret curves.
2. Sweep ε and the UCB exploration constant; observe the exploration/ exploitation tradeoff.
3. Make the bandit non-stationary; show fixed-step-size ε-greedy tracking drift where sample-averaging fails.

2. The goal

A bandit is a Markov decision process with the state stripped out. There is one situation, k actions, and a reward drawn from a fixed distribution per action. No transitions, no credit assignment across time, no discounting.

That emptiness is the point. Every hard thing in RL — bootstrapping, function approximation, delayed reward — is absent, so the one remaining difficulty stands alone: **you can only learn about an action by taking it, and every step spent learning is a step not spent earning.** That is the exploration/exploitation tradeoff, and A1 exists to make it concrete before anything else is layered on.

Three ideas installed here reappear for the rest of the ladder:

- **Regret as the scoring function.** Not "did it converge" but "how much reward did the learning cost". This is the frame for every ablation later.
- **Optimism and posterior sampling as exploration strategies.** UCB's confidence bonus is the ancestor of the exploration terms in A3–A6; A6's MCTS selects child nodes with a UCB rule (UCT) that is literally UCB1 applied to a search tree.
- **Tracking vs. averaging.** Whether an estimator should weight all history equally or forget. This decides the target-network and replay-buffer questions in A3, and the KL-to-reference question in B1.

3. The problem setup

The standard Sutton & Barto ch. 2 testbed, $k = 10$:

- Each arm has a hidden true value $q^*(a) \sim \mathcal{N}(0, 1)$, drawn once per run.
- Pulling arm `a` returns $r \sim \mathcal{N}(q^*(a), 1)$.

The agent sees only realized rewards, never `q*`. It must infer which arm is best from noisy samples whose noise (σ=1) is the same order as the spread between arms (σ=1) — so a single pull tells you almost nothing, and the difficulty is real.

Formally. A k-armed bandit is the tuple $(\mathcal{A}, \{P_a\}_{a \in \mathcal{A}})$ with $|\mathcal{A}| = k$ and $q^*(a) = \mathbb{E}_{r \sim P_a}[r]$. Write $a^* \in \arg\max_a q^*(a)$ and the *gap* of an arm as $\Delta_a = q^*(a^*) - q^*(a) \geq 0$. A policy is a rule mapping the history $h_{t-1} = (a_1, r_1, \ldots, a_{t-1}, r_{t-1})$ to a distribution over $\mathcal{A}$.

Metric. We score by *expected* cumulative regret:

$$\text{regret}(T) = \sum_{t=1}^{T} \left[\max_a q^*(a) - q^*(a_t) \right]$$

Note it uses q^*, the hidden truth, not the realized reward. An agent that picks the best arm and gets unlucky noise incurs zero regret, correctly — it made the right decision. Scoring on realized reward would add σ=1 noise per step to every curve and measure luck alongside skill. This is the single most useful methodological choice in the phase, and it is only available because the testbed is synthetic: from A3 onward the true optimum is unknown and we are forced back onto noisier proxies.

4. The three algorithms

All three keep a value estimate `Q(a)` per arm and differ *only* in how they decide what to try next.

ε-greedy — explore by fixed random tax

With probability ε pick a uniformly random arm, otherwise pick $\arg\max_a Q(a)$.

Estimates update incrementally, $Q(a) \leftarrow Q(a) + \alpha\,[r - Q(a)]$, with $\alpha = 1/N(a)$ giving the exact running mean.

Simple and hard to break, but the exploration never adapts. After 10 000 pulls, when the best arm is obvious, an $\varepsilon = 0.1$ agent still throws away 10% of its pulls — and it spreads them uniformly, sampling an arm it has already ruled out just as often as a plausible contender.

UCB1 — explore by optimism under uncertainty

Pick the arm maximizing

$$Q(a) + c\sqrt{\frac{\ln t}{N(a)}}$$

The bonus is a confidence-interval width: by Hoeffding's inequality the error of a mean over N samples shrinks like $1/\sqrt{N}$, and the $\ln t$ numerator keeps the bound valid as the number of decisions grows. So the rule is **act as if each arm is as good as its evidence permits.** An arm is then tried either because it looks good (high Q) or because it is under-explored (few N) — and the bonus decays automatically as evidence accumulates. No wasted pulls on arms already known to be bad.

Thompson sampling — explore by posterior sampling

Keep a posterior over each arm's mean, draw one sample per arm, act greedily on the draws. With a Gaussian prior $\mathcal{N}(0, \sigma_0^2)$ and Gaussian noise $\mathcal{N}(\theta, \sigma^2)$, the conjugate posterior after n pulls summing to s is Gaussian with

$$\tau = \frac{1}{\sigma_0^2} + \frac{n}{\sigma^2}, \qquad \mu = \frac{s}{\sigma^2 \tau}, \qquad \text{variance} = \frac{1}{\tau}$$

An arm is then chosen exactly as often as it is *probably* optimal — probability matching. Exploration falls out of posterior width rather than being bolted on, so it shrinks by itself as the posteriors sharpen.

What is actually guaranteed

Writing $N_T(a)$ for the number of pulls of arm a in T steps, regret decomposes exactly as

$$R(T) = \sum_a \Delta_a \, \mathbb{E}[N_T(a)]$$

so every bound below is really a statement about how often a suboptimal arm is pulled. Three results frame what the experiments should show.

1. Fixed ε gives linear regret. An ε-greedy policy explores uniformly with probability ε at every step, so for any suboptimal a, $\mathbb{E}[N_T(a)] \geq \varepsilon T/k$ regardless of how much it has already learned. Hence

$$R(T) \geq \frac{\varepsilon}{k} \sum_{a \neq a^*} \Delta_a \cdot T = \Omega(T)$$

The constant is small, but the *growth rate* is linear — the exploration tax is paid at a fixed rate forever. Any algorithm with sublinear regret must let its exploration decay.

2. UCB1 achieves logarithmic regret. For rewards bounded in $[0,1]$, Auer et al. (2002) prove

$$R(T) \le \sum_{a:\Delta_a>0} \frac{8\ln T}{\Delta_a} + \left(1+\frac{\pi^2}{3}\right)\sum_a \Delta_a$$

Proof sketch. A suboptimal a is pulled at step t only if its index exceeds the optimal arm's, which requires at least one of three events: the optimal arm's mean is badly underestimated, a's mean is badly overestimated, or a has been pulled few enough times that its confidence term alone covers the gap, i.e. $N_t(a) < 8\ln T/\Delta_a^2$. Hoeffding's inequality bounds the first two events by t^{-4} each, and summing over t gives the $\pi^2/3$ constant; the third contributes the $8\ln T/\Delta_a$ term directly. The $\ln t$ inside the bonus is exactly what makes the Hoeffding union bound hold uniformly over time.

Two honest caveats for this repo's setup: the testbed uses Gaussian rewards rather than $[0,1]$-bounded ones, so the constant does not transfer directly; and the implementation uses a tunable c where standard UCB1 fixes $c=\sqrt{2}$. The measured optimum of $c=1$ is therefore *below* the theoretical setting — consistent with the bound being worst-case over gap configurations while the sweep optimises for one particular horizon.

3. Nobody can do better than logarithmic. The Lai–Robbins lower bound says that for any policy that is consistent (sub-polynomial regret on every instance),

$$\liminf_{T\to\infty} \frac{R(T)}{\ln T} \ge \sum_{a:\Delta_a>0} \frac{\Delta_a}{\mathrm{KL}(P_a \,\|\, P_{a^*})}$$

So UCB1 is order-optimal, and the gap between it and Thompson sampling in the experiment is a matter of constants, not rates. Thompson sampling attains the same $O(\ln T)$ rate and, for several standard families, matches the Lai–Robbins constant asymptotically; its Bayesian regret is $O(\sqrt{kT\ln T})$.

Why Thompson sampling explores correctly. By construction it draws θ_a from the posterior and acts greedily, so

$$\Pr(\text{select } a \mid h_t) = \Pr\left(a \in \arg\max_{a'} \theta_{a'} \;\middle|\; h_t\right) = \Pr(a = a^* \mid h_t)$$

an arm is played exactly as often as the posterior believes it is optimal. That is *probability matching*, and it is why exploration decays automatically: posterior mass concentrates, so the probability of drawing a suboptimal arm falls without any schedule being specified.

Why item 3 works: the update rule *is* the memory

Unrolling the incremental update shows what each choice of `α` remembers:

$$Q_{n+1} = (1-\alpha)^n Q_1 + \sum_{i=1}^{n} \alpha(1-\alpha)^{n-i} R_i$$

With **constant** α, weights decay geometrically with age — an exponential recency average that forgets. With $\alpha = 1/N(a)$, every reward gets weight $1/n$: the reward from pull 1 counts exactly as much as the reward from pull 9 999.

On a stationary bandit that even weighting is optimal — it is the minimum-variance estimator of a fixed mean. On a drifting one it is a liability, because the old rewards describe a world that no longer exists. Item 3 makes an agent pay for that.

The rigorous statement is a stochastic-approximation one. Robbins–Monro convergence of $Q_n \to q^*$ requires

$$\sum_{n=1}^{\infty} \alpha_n = \infty \qquad \text{and} \qquad \sum_{n=1}^{\infty} \alpha_n^2 < \infty$$

The first condition says the steps are large enough in aggregate to reach any starting point; the second says they shrink fast enough for the noise to average out. Sample averaging, $\alpha_n = 1/n$, satisfies both — $\sum 1/n$ diverges and $\sum 1/n^2 = \pi^2/6$ — so it converges almost surely to a fixed q^*.

A constant α satisfies the first and **fails the second**: $\sum \alpha^2 = \infty$. So it provably does *not* converge to a point. The estimate instead keeps fluctuating, with asymptotic variance $\frac{\alpha}{2-\alpha}\sigma^2$ for observation noise σ^2.

That failure is the feature. Convergence to a point is precisely what you do not want when the point is moving: the same property that disqualifies constant α under the Robbins–Monro theorem is what lets it track. The two agents in item 3 are a demonstration that "converges" and "is useful" can be opposites.

5. What was done

Code in `experiments/a1_bandits/`: `bandits.py` holds both testbeds, the three agents, and `run_episode`; three scripts run the three items.

Design decisions worth stating:

- **100 seeds per configuration.** Bandit runs are extremely high-variance — a single run's regret is dominated by which arm happened to look good first. The protocol's floor is 3 seeds; 100 costs seconds here and is what makes the variance findings below trustworthy.
- **Every plot is a mean ±1 std band across seeds**, never a single run.
- **Ties in argmax are broken uniformly at random.** With zero-initialized estimates all k arms tie on step 1, so `np.argmax` would deterministically open on arm 0 for every agent and every seed — a small bias that would quietly contaminate all three items.
- **The drifting testbed starts all arms equal** (`q* = 0`) and random-walks each with σ=0.01 per step, per S&B exercise 2.5, so there is no initial signal to memorize and the entire task is tracking.
- Item 3's two agents share the same ε and differ *only* in the update rule, so the comparison isolates estimator memory rather than exploration.

6. What came out

Ranking (1000 pulls): Thompson 71.5 ± 33.0 regret, UCB1(c=2) 153.0 ± 33.2, ε-greedy(0.1) 226.2 ± 131.4. The ordering matches the theory — adaptive exploration beats a fixed tax.

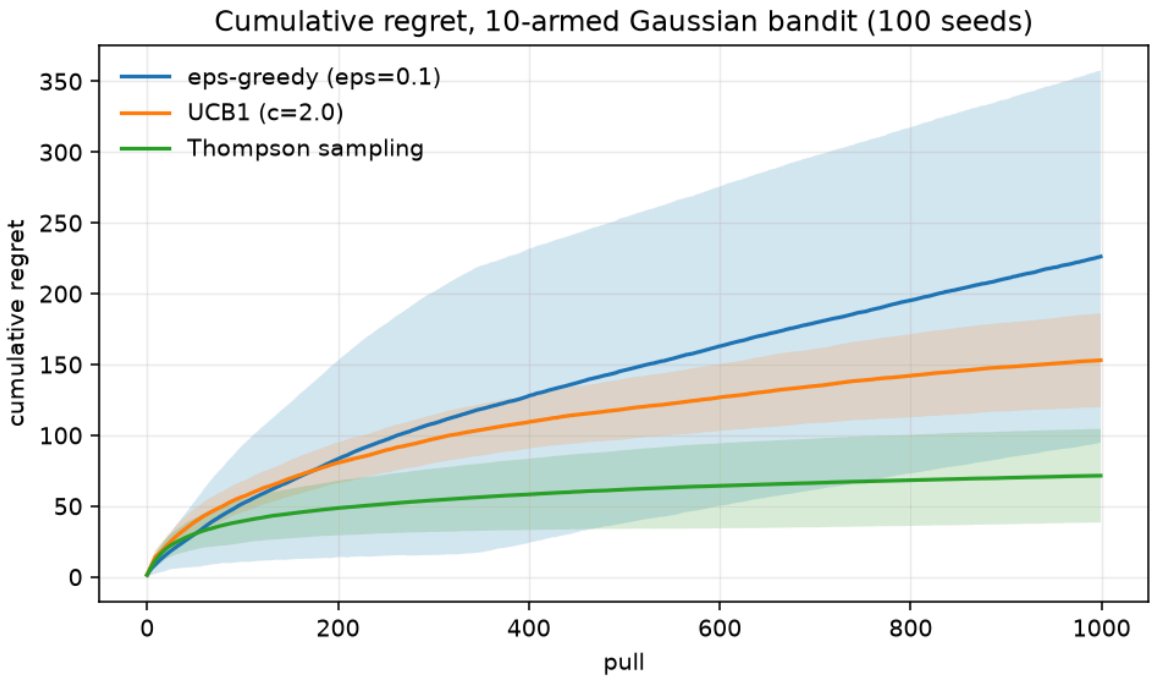


Figure — Cumulative regret on a 10-armed Gaussian bandit, 100 seeds, ±1 s.d. bands. Thompson sampling is lowest and has the narrowest band.

Both sweeps are cleanly inverted-U, which is the tradeoff made visible: too little exploration and you never find the best arm, too much and you keep paying after you have. The optima are ε=0.04 and c=1.0.

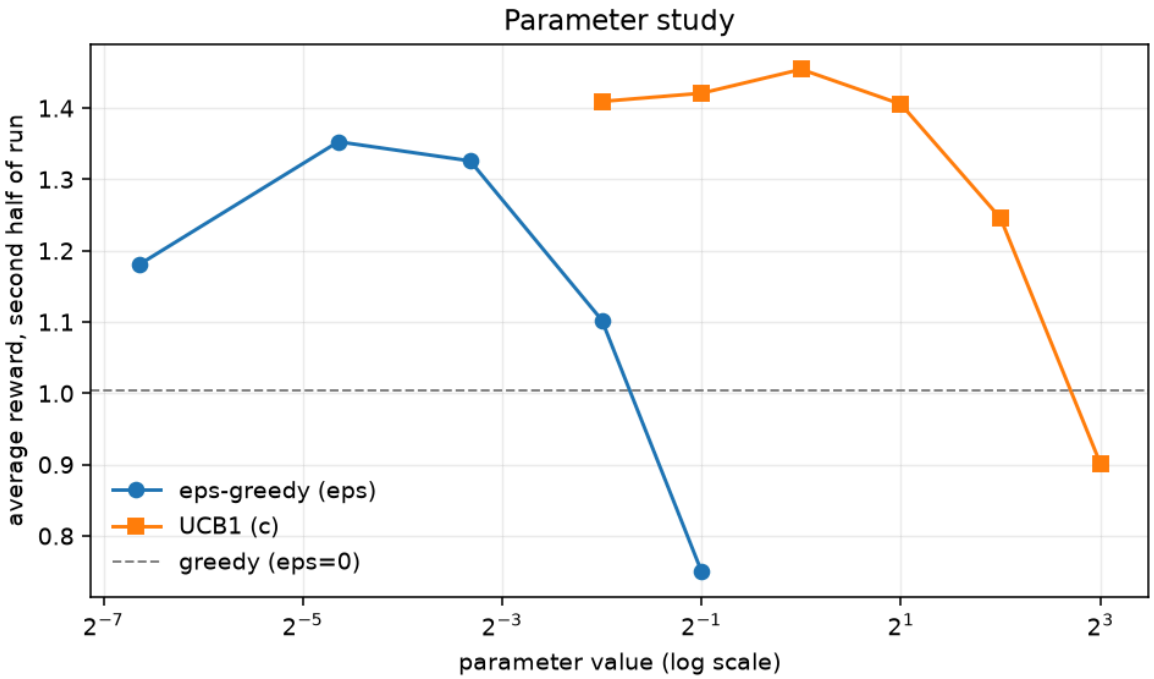


Figure — Average reward over the second half of the run against the exploration parameter, on a log scale: ε for ε-greedy, c for UCB1. The dashed line is pure greedy.

c=2 — the constant usually quoted as standard — is more than twice as costly as c=1 here (153 vs 62 regret). That is not UCB being wrong; it is horizon dependence. A larger `c` front-loads exploration that only amortizes over a longer run than 1000 pulls. The lesson generalizes: an exploration constant is only meaningful relative to a budget.

Under-exploration is punished too, not just over-exploration. ε=0.01 (359) does worse than ε=0.1 (226) despite being nearer the asymptotically better setting — at 1000 pulls it simply has not identified the best arm yet. Pure greedy is worst at 484.

Thompson's advantage is as much variance as mean (±33 vs ε-greedy's ±131). ε-greedy's exploration is a fixed random tax that cannot adapt to a run going badly, so seeds diverge; Thompson's shrinks as posteriors sharpen. A comparison of mean regret alone would have missed this — a warning for reading single-curve figures in papers later.

The non-stationary result is stark: constant α reaches 74% optimal actions against sample averaging's 45%, with 2.2× less regret.

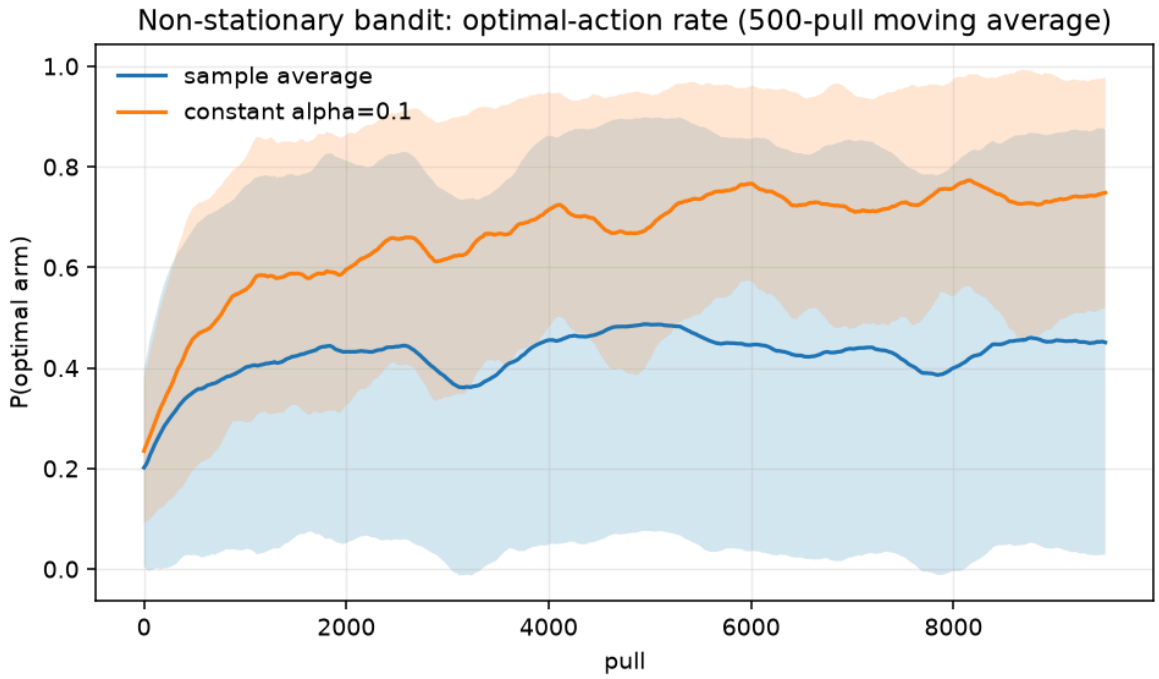


Figure — Drifting bandit: fraction of pulls on the current best arm (500-pull moving average). The sample-average band spans nearly the whole range, which is the stale-information failure.

And the most useful finding is in the band, not the mean. The sample-average agent's spread covers nearly the whole 0–1 range (final regret std 1687 vs 194). It does not fail uniformly — it commits to whichever arm looked best early, then either gets lucky as the walk favours it or never recovers. That is a **bias toward stale information, not variance**, and no number of extra seeds fixes it. Averaging over seeds hides it; the band is what exposes it.

7. What this sets up

- **A2** faces the same `α` choice for TD updates, now with a moving bootstrap target — which makes the environment non-stationary *from the learner's point of view* even when the MDP is fixed.
- **A3**'s replay buffer is exactly the stale-information failure from item 3, in a setting where the drift is caused by the agent's own improving policy.
- **A6**'s MCTS selects nodes by UCT — UCB1 applied to a search tree.
- **B3**'s GRPO replaces a learned value baseline with the mean over a group of rollouts, which is a bandit-style estimate of an action's relative worth; the group-size sweep there is the direct descendant of item 2's sweeps.

8. Where these items come from

Every item traces to a specific piece of source material, so the curriculum is auditable rather than a list someone felt like writing. Section numbers follow the published 2nd edition of Sutton & Barto; the free 2017 draft shifts chapter 2 by one section (its §2.4 is the published §2.5) and numbers the non-stationary exercise 2.3 rather than 2.5.

item	primary source	why it was picked
1 — implement the three agents, plot regret	S&B §2.2–2.4 (action-value methods, the 10-armed testbed, incremental implementation); §2.7 for UCB; Auer, Cesa-Bianchi & Fischer 2002 for the UCB1 bound; Russo et al. 2018 tutorial for Thompson sampling	The three algorithms are the three *distinct answers* to exploration — fixed randomization, optimism, posterior sampling. Any two of them would leave a strategy unrepresented.
2 — sweep ε and c	S&B's parameter study, the summary figure of ch. 2 (draft Fig. 2.6), which plots every bandit algorithm's reward against its parameter on a log axis	The inverted-U is the tradeoff made visible. It is also the figure this repo's `parameter_study.png` deliberately reproduces.
3 — non-stationary tracking	S&B §2.5 "Tracking a Nonstationary Problem" and its programming exercise (Exercise 2.5 in print, 2.3 in the draft), which specifies the random-walk testbed used here	It is the one bandit result that generalizes directly to deep RL, where the target is always moving.

The **lab** pairing (`Practical_RL` week05_explore) is the ladder's own choice, and it covers exactly these three algorithms — which is why A1 is the phase it is attached to.

The **goal statement** in section 2 is the ladder's, quoted from `ladder.md`: *"the exploration/exploitation tradeoff in its purest form, before state or function approximation get added."* The forward links in section 7 are this repo's reading of how the phases connect, not claims from any source.